

Amber — Design

Status: draft for sign-off — 2026-07-02

What Amber is

Amber is a **headless personal knowledge hub** (a "second brain") for one person running multiple businesses. It has two regimes joined by one link graph:

- a **rigid spine** — development tracking, Monday-model: boards of initiatives with statuses, the anchor everything connects to;
- a **fluid sediment** — ideas, brainstorm sessions, content: stateless, trail-based, never deleted.

N conversational clients (Claude app, Hermes, future web/mobile chat) and pipelines (n8n) read and write through one tool surface. Amber itself does not converse.

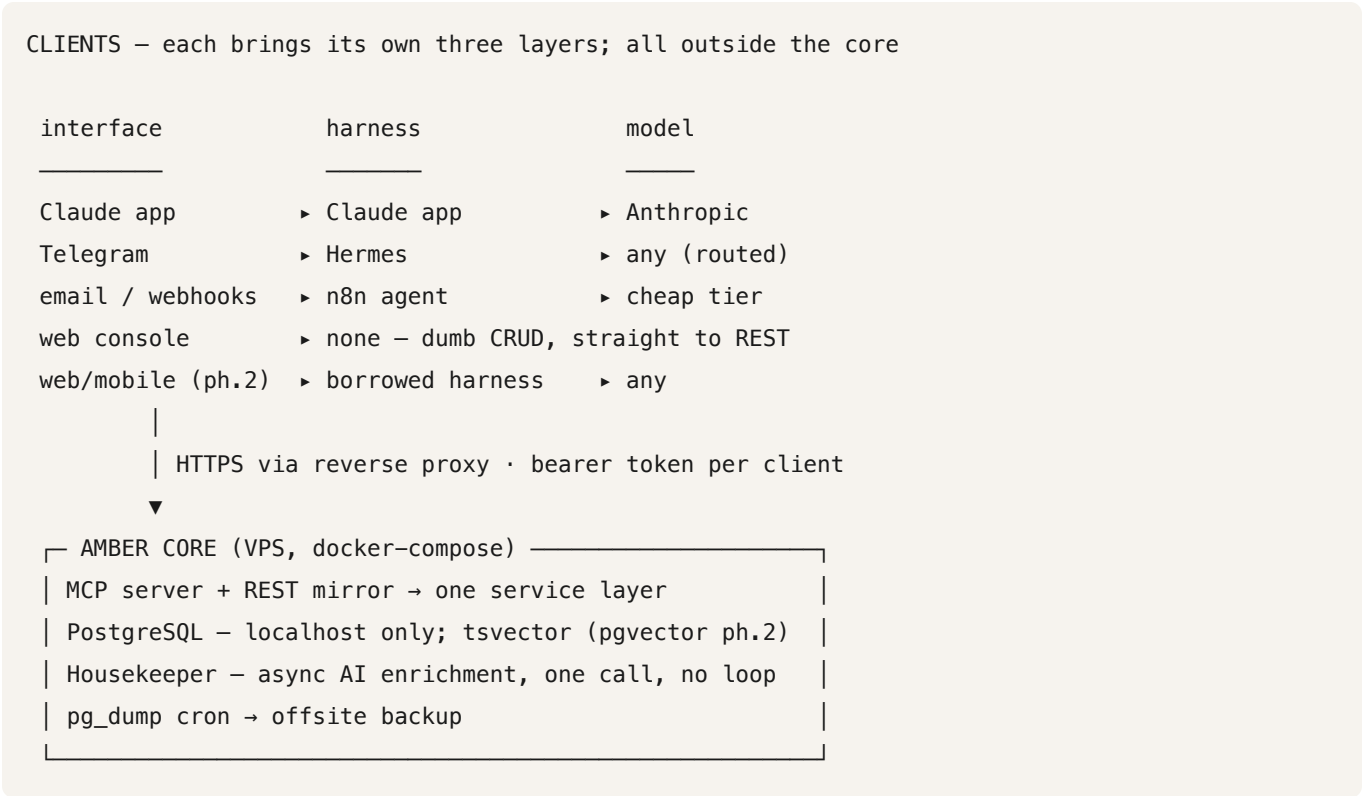
Guiding metaphor, made literal: declared state is dangerous for thoughts and correct for work. Ideas flow; initiatives have statuses. Nothing is ever deleted — things sink and can resurface.

Core principles

1. **Intelligence at the edge.** Conversational intelligence lives in clients. Channels deliver *structured* data; dumb channels get an adapter (n8n agent) on the channel, never in the core. Sole exception: the Housekeeper.
2. **One tool surface, n clients.** The tool surface is the product. No Amber feature may ever assume a specific client. Interface / harness / model are three separately swappable layers, all outside the core. Amber never builds a harness — it borrows one (Hermes engine first choice, n8n agent fallback, thin SDK harness last resort).
3. **Nothing is ever deleted.** Entities can be **sunk** (faded from foreground) but stay in the corpus forever — searchable, embeddable, resurfaceable. The promise is also about loss windows, not just delete paths: target **RPO ≤ 5 minutes** via WAL archiving / point-in-time recovery, with restore drills — a backup that has never been restored is a hypothesis. One sanctioned exception to permanence: **break-glass redaction** — transcript/content bodies can be scrubbed (row and audit trail preserved) for PII or leaked secrets, human-initiated only.
4. **Flowful thinking layer.** Ideas have no status column. Condition (warm/dormant, shallow/deep) is derived at read time from the activity trail. The only manual lifecycle gesture is sink. (*Honesty note: until pgvector lands in phase 2, resurfacing sunk material depends on full-text keywords you remember — phase-1 "resurface" is deliberately weak.*)
5. **Store text cleanly for the future.** All text in Postgres with tsvector now; reserved path to pgvector in phase 2. Discovered links are purely additive later.

6. **Corpus-derived text is inert.** Everything extracted from untrusted sources — transcripts, PDF/email text, machine-written summaries and tags — is served at every boundary as provenance-labelled *data*, never as instructions. No client may justify a write by citing transcript- or extract-origin text without human confirmation; this rule ships as a mandatory line in every client's system configuration. This closes the laundering path: malicious content → catalogue → `get_context` → a tool-holding agent.

Architecture



- **Per-client tokens** — hashed, scoped, individually revocable; every row records its writing client, so "every writer identifiable" is a data-model fact. (*Verified limitation: claude.ai custom connectors don't support bearer headers — the Claude app authenticates via a dedicated token embedded in its connector URL path, registered as a no-auth connector; MCP OAuth is the documented upgrade path.*)
- **REST mirror** of the tool surface for non-MCP clients (n8n, console). One service layer under both.
- Single database; every entity carries `workspace` (`natives` | `meanwhile` | `personal`).

Clients: interface / harness / model

Every conversational client decomposes into three separately swappable layers, all outside Amber core:

- **Interface** — where the conversation happens: Telegram, the Claude app, a web page, a mobile app.
- **Harness** — the agent runtime: the loop, tool-call plumbing, model routing, conversation state, retries. This is the expensive part, and Amber never builds one.

- **Model** — whatever brain the harness routes to.

Current lineup: the **Claude app** bundles all three (interface + harness + Anthropic models) and speaks remote MCP — zero build. **Hermes** ([NousResearch/hermes-agent](https://nousresearch.com/hermes-agent), already running) is an existing harness with a Telegram interface and free model choice — **source-verified, not yet live-tested**: its MCP client documents remote Streamable-HTTP servers with bearer-token headers (`tools/mcp_tool.py`), and its `on_session_finalize` plugin hook + SQLite transcript store (`load_transcript`) allow a small plugin to POST the transcript to `save_session` at session end, making capture structural rather than model-remembered. Because two phase-2 bets (web chat on Hermes's engine, proactive routines) rest on this, an **end-to-end smoke test** — live tool call from Telegram against a probe endpoint behind the reverse proxy — is the exit gate of build step 2, before anything is built on top. **n8n** carries a rudimentary harness (AI agent nodes, MCP client support) — good enough to be the intelligence adapter for dumb channels (email-to-Amber, webhooks) and an emergency fallback.

Phase 2 — Amber-hosted chat (web/mobile): when the web console grows a chat box, the agent behind it is *just another client of the core*, even if it runs in the same docker-compose — same tokens, same tools, no privileged access. Harness candidates in order: **Hermes's engine** with a web interface bolted on (first choice — it exists and is already pointed at Amber's tools), **n8n's agent runtime** (free, already deployed, functional but unglamorous), or a **thin harness built on an agent SDK** (last resort: days to build, yours to maintain forever). Interface candidates are decoupled from that choice: a custom web front on Hermes's engine, or an off-the-shelf self-hosted chat UI (LibreChat / Open WebUI — interface-first products whose built-in harness is shallow but may suffice for "chat with tools").

Because the tool surface is client-agnostic, none of these choices is a marriage: swapping harnesses means re-pointing a different client at the same endpoint.

The partnership (the hybrid intuition). Amber and Hermes are two halves of one system, deliberately on **separate servers as a trust boundary** (Amber beside the business data; Hermes on its own agent sandbox, speaking to Amber over HTTPS like any client): Amber is the memory layer — dumb, permanent, self-sufficient; Hermes is the voice and initiative layer — smart, replaceable. Hermes's harness makes Amber smart in three escalating ways:

1. **Reactive (phase 1)**: all conversation with the corpus runs through Hermes's harness (Telegram now, web/mobile adapters on its gateway later).
2. **Proactive (phase 2)**: Hermes's native cron/routines call Amber's tools on schedule — a morning digest across boards delivered on Telegram, staleness nudges, resurfacing of sunk-but-resonant ideas. This is the launchpad in conversational form, for near-zero build.
3. **Curatorial (phase 2) — the Librarian**: a nightly Hermes routine with full tool access roams the corpus and *proposes* discovered links, duplicate merges, and resurrections — advisory only, confirmed by Antonio in chat or console; a background agent never silently rewrites memory.

The boundary that keeps the partnership safe: **Hermes makes Amber eloquent; it is never load-bearing for Amber's memory**. Data-integrity duties (capture, storage, the Housekeeper's guaranteed enrichment)

live in the core and function with every client dead. If the Librarian misses a night, quality dips; nothing is lost.

The spine (development tracking, Monday-model, single-user)

- **board** — belongs to a workspace. Has configurable **status set** and **lane set** (labels + colors). Typically one initiatives board per workspace or domain; grouping in the UI is by status.
- **item (initiative)** — a work-entity (feature, supplier deal, campaign), *not* a task. Fixed column set, from the real board in use today: name, lane, priority, status, progress, due date, note — plus `extra JSONB` as the escape hatch for future column needs (no migration required to experiment).
- Attachments = content linked to the item. "Connected to" = link table. External ref IDs (Todoist/Linear/Gmail) stored inert on items/projects until phase 3.
- Day-to-day tasks stay in Todoist; engineering tickets stay in Linear. The spine tracks initiatives; phase 3 joins execution detail into the launchpad via refs.

The fluid layer (thinking)

- **idea** — title, description, workspace, optional board-item or board anchor. No status. `sunk_at` is the only lifecycle field. Condition derived from the trail.
- **session** — one conversation/ramble/note: type (voice/chat/note), capturing client, raw transcript (archival, searchable), Housekeeper handoff summary.
- **content** — links, PDFs, emails, notes: extracted full text (extraction is the channel's job — n8n, console paste, or the agent in-conversation), source metadata, Housekeeper catalogue (summary, tags, suggested home), tsvector.
- **link** — generic typed edge (`from`, `to`, `type`, `note`, `created_at`) across *all* entities, both regimes. A session-touches-idea link carries the per-idea note: *where the thinking stopped, declared next step*. An idea's trail = its touch links ordered by time.
- **Graduation** — `promote_idea` : an idea that becomes work spawns a spine item, keeping its links to every session and content piece that formed it. (The "SPEC" status living awkwardly in the current tracking tool becomes this seam.)

Two kinds of connection, forever distinct: **declared links** (facts, phase 1) and **discovered links** (semantic resonance via pgvector, phase 2 — computed at read time, never stored as facts).

Capture flow (decided: live extraction)

Mid-conversation the client agent makes **live tool calls** — `save_idea`, `touch_idea`, `update_item`, `link_items` — facts land immediately, confirmed in-conversation. The client also has live *read* access (`get_context`, `search`) so it can spar with the corpus while you ramble. At session close, the client calls `save_session` with the transcript, which enqueues the Housekeeper.

Session-close asymmetry (accepted limitation). Clients are not equal here. Hermes closes sessions *structurally*: its `on_session_finalize` hook fires from the gateway's expiry watcher and a small plugin POSTs the transcript — guaranteed, no model judgment involved. The Claude app has no such hook: `save_session` there is **model-remembered** — it happens only if the model follows a fixed instruction in the Claude project ("always end by calling `save_session`"), which principle 1 would normally forbid relying on. Accepted because live extraction bounds the damage: ideas and touches land *during* the conversation, so a forgotten close loses at most the transcript and its handoff — never the facts. The convention must be documented in the README's Claude-app setup section.

The Housekeeper

The core's single AI worker: async, non-critical-path, no harness, no loop, no tools — one prompt-to-completion call per job on a cheap model (IDs verified online at implementation time, never from training data).

- On `save_session` → **handoff summary**, compaction-style: what we were doing, what was decided, what stayed open, what's next.
- On `save_content` → **catalogue**: short summary, topic tags, suggested workspace/project home.

Context-stuffed, not agentic: before each call, Amber assembles relevant corpus context *deterministically* (SQL, not an agent): the workspace's boards/items, warm ideas with latest touch notes, recent handoffs, tag vocabulary, full-text matches against the input. The Housekeeper distills *against what Amber already knows* — recognizing that "the export thing" is an existing idea, that a PDF belongs beside its siblings — while remaining a single guaranteed completion. Same mechanism as `get_context`, pointed inward.

Writes are never blocked by it; failures retry. One tuned prompt per job → uniform quality regardless of capturing client. Expected cost: cents per month.

The resume readback (crown jewel)

`get_context(anchor)` — for a board, item, or idea — returns compact, LLM-optimized structure: warm ideas with latest touch notes (where each stopped, next steps), relevant spine items with status, recent session handoffs, linked content refs; sunk items excluded by default. The client narrates: "*Last touched Tuesday. Fatture is in corso. Three warm threads: export feature — torn between A and B, leaning A, next step test on real data; pricing — waiting on the supplier email; a fresh spark about notifications, one mention.*"

Foreground/background is derived: recency and density of trail activity. Sunk items surface only via explicit search or (phase 2) semantic discovery.

Tool surface v1

Spine: `list_boards` · `get_board` · `save_item` · `update_item` · Fluid: `save_idea` · `touch_idea` · `promote_idea` · `save_session` · `save_content` · Both: `get_context` · `link_items` · `sink_item` · `search` (full-text, cross-entity, includes sunk)

Documented well enough that any harness can be pointed at it cold. REST mirror for non-agent writers.

MVP scope and build order

The spine is the foundation — links need anchors — so its *schema* lands first. The UI does not: Amber's differentiating value is the brainstorm→persistence→resume loop, and boards already exist (ugly but functional) in ClickUp. Capture ships before console.

1. **Schema + migrations** — both regimes, one DDL, tsvector indexes.
2. **API core, deployed** — service layer, MCP server + REST mirror, per-client bearer tokens, live behind the existing reverse proxy (minimal deploy — clients need HTTPS to connect at all). **Exit gate: end-to-end smoke test** — a live tool call from Hermes-on-Telegram and from the Claude app against a probe tool. This converts the source-level Hermes compatibility check into proof before anything is built on top of it.
3. **Fluid capture** — idea/session/content tools live; Hermes connected (MCP config + `on_session_finalize` plugin); Claude app connected (remote MCP + fixed project instruction); `get_context` and `search` serving.
4. **Housekeeper** — handoffs + cataloguing; completes the resume loop.
5. **Web console v1 — the board** (replaces the ClickUp board): grid grouped by status, inline editing, the screenshot's columns — designed with real captured data already in the system. No drag-and-drop, no extra views initially.
6. **Console v2 — trust & repair**: browse/fix ideas, sessions, content, links; global search.
7. **Ops hardening** — `pg_dump` offsite cron, README, backup restore drill. The drill is pinned to the data, not the build order: first successful restore within the first month of real captures, then recurring — a backup that has never been restored is a hypothesis, not a backup.

Out (explicitly): pgvector / discovered links (phase 2 headline) · launchpad view (fast-follow, designed from real captured data) · Amber-hosted chat (phase 2+, borrowed harness) · dumb-channel adapters (n8n, phase 2) · Todoist/Linear/Gmail integration (phase 3; refs stored inert) · custom column engine (JSONB escape hatch until proven need) · multiple board views, automations, multi-user — not for now.

Deferred to implementation plan

- Stack pick (Node/TS + official MCP SDK vs Python FastMCP), argued briefly; leaning TS (mature remote-MCP support, one language with the console).
- Reverse proxy inspection on the VPS (nginx or caddy) and deployment wiring.
- Session-close conventions per client.
- Housekeeper queue mechanism (Postgres queue vs cron sweep — no new infrastructure either way).

- Console UI stack.
- Migration of the existing ClickUp initiatives board (the column-set reference) into the first Amber board — **including completed/closed items as corpus**, with their terminal status: "nothing is ever deleted" applies to the migration too, and shipped history becomes searchable (and, in phase 2, embeddable) memory.